

Pose and Homogeneous Transforms

A Primer for the UR5 Assignment

About this primer. This is a compact, self-contained starting point so you can begin the UR5 assignment before the topic is covered in full during lectures. **Part A** introduces pose, the homogeneous transform, and the small amount of notation and matrix algebra you actually need. **Part B** applies those tools to the assignment: using points whose coordinates are known in two frames to find where a piece of equipment sits relative to the robot, and then computing the pose to send the robot's tool to in order to complete a task.

The worked examples here are kept planar (a translation plus one rotation about z) even though Part A is written for full three-dimensional pose; the general case is developed in lectures. The notation follows Corke, *Robotics, Vision and Control* (2nd ed.), if you want to read further.

Before you start. Read the assignment brief and open its points spreadsheet first, so the frames and key-numbers in this document make sense.

Part A: Background

Pose: coordinate frames in three dimensions

1 What is pose?

The *pose* of a rigid body is its position and orientation relative to some reference frame. In three dimensions we describe pose by attaching a coordinate frame $\{B\}$ to the body and expressing that frame relative to a reference frame $\{A\}$ using two pieces of information:

- a **translation** ${}^A t_B = (t_x, t_y, t_z)$, the position of the origin of $\{B\}$ expressed in $\{A\}$'s coordinates;
- a **rotation** ${}^A R_B \in SO(3)$, a 3×3 orthonormal matrix describing the orientation of $\{B\}$'s axes relative to $\{A\}$'s axes ($SO(3)$ is just the name for the set of all such rotation matrices).

Together, $({}^A R_B, {}^A t_B)$ fully define the pose of $\{B\}$ with respect to $\{A\}$.

2 The homogeneous transform

Keeping R and t as two separate objects works, but it makes composing frames awkward: rotating and translating have to be applied as two separate steps, in the right order, every time. Packing them into a single 4×4 **homogeneous transform**

$${}^A T_B = \begin{bmatrix} {}^A R_B & {}^A t_B \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

turns that bookkeeping into ordinary matrix multiplication. The blocks are sized so this works out: ${}^A R_B$ is 3×3 , ${}^A t_B$ is a 3×1 column, and the bottom row is $0 \ 0 \ 0 \ 1$, three zeros rather than four because ${}^A t_B$ takes up a single column.

If a third frame $\{C\}$ (say, a tool mounted on the robot) is known relative to $\{B\}$, and $\{B\}$ is known relative to $\{A\}$, then the pose of $\{C\}$ relative to $\{A\}$ is

$${}^A T_C = {}^A T_B {}^B T_C$$

This one property, that composing poses is just matrix multiplication, is the entire reason homogeneous transforms, rather than a rotation matrix and a translation vector kept separately, are the standard way to represent pose in robotics.

3 Reading the notation

Every homogeneous transform carries two frame labels, ${}^A T_B$, read as “the pose of $\{B\}$, expressed in $\{A\}$ ”. The two labels are not interchangeable:

A (leading superscript): the **reference frame** (parent frame), the frame the numbers are expressed in.

B (trailing subscript): the **target frame** (child frame), the frame whose pose is being described.

The same convention applies to R and t on their own, and to the coordinates of a point, p : ${}^A R_B$, ${}^A t_B$ and ${}^A p$ all use a leading superscript for the frame the numbers are expressed in. Note: throughout, a triple written out in line, such as (t_x, t_y, t_z) , is a column vector; it is written as a tuple only to save space.

Chaining transforms

Chains are built by **left-multiplying**. Start with the transform for the frame you actually care about, relative to its immediate parent: here ${}^B T_C$, describing $\{C\}$ relative to its parent $\{B\}$. To extend the chain out towards the overall reference frame $\{A\}$, left-multiply by the next transform up the hierarchy, ${}^A T_B$:

$${}^A T_C = {}^A T_B {}^B T_C$$

A longer chain just repeats the pattern, working outward one parent at a time. For example, if $\{D\}$ ’s immediate parent is $\{C\}$, start at ${}^C T_D$ and left-multiply twice, out to $\{A\}$:

$${}^A T_D = {}^A T_B {}^B T_C {}^C T_D$$

Left-multiply, always. Matrix multiplication does not commute:

$${}^A T_B {}^B T_C \neq {}^B T_C {}^A T_B$$

and the second ordering is not even a valid chain, since adjacent frames don’t match (see below). Always build a chain by starting at the child end, the frame you actually care about relative to its immediate parent, and repeatedly left-multiplying by the next transform up the hierarchy, one parent at a time, until you reach the overall reference frame.

A quick way to check that a chain has been written correctly: for every adjacent pair of transforms in the product, the target (subscript) of the left one must match the reference (superscript) of the right one. These matching inner labels sit next to each other and cancel, leaving only the overall reference and target behind:

$$\begin{matrix} A & & & B \\ T & = & T & T \\ C & & B & C \end{matrix}$$

The adjacent **B**’s cancel, leaving just the outer A (reference) and C (target).

Inverting a transform

Swapping which frame is the reference and which is the target inverts the transform: describing $\{A\}$ relative to $\{C\}$, given ${}^A T_C$, is just the matrix inverse,

$${}^C T_A = ({}^A T_C)^{-1}$$

This is not a separate computation to memorise; it drops straight out of composing a transform with its own reverse: ${}^A T_C {}^C T_A = {}^A T_A = I$ (a frame expressed in itself is just the identity), so ${}^C T_A$ must be whatever matrix undoes ${}^A T_C$, i.e. its inverse.

In practice you never need to invert a general 4×4 matrix by hand: because R is orthonormal, $R^{-1} = R^\top$, so

$${}^C T_A = ({}^A T_C)^{-1} = \begin{bmatrix} ({}^A R_C)^\top & \overbrace{({}^A R_C)^\top}^2 \overbrace{(-{}^A t_C)}^1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The translation term does two things to ${}^A t_C$, applied right to left:

1. $(-{}^A t_C)$ **reverses the arrow**. ${}^A t_C$ points from $\{A\}$'s origin to $\{C\}$'s origin, but ${}^C T_A$ describes $\{A\}$ as seen from $\{C\}$, so it needs the opposite arrow, from $\{C\}$'s origin back to $\{A\}$'s.
2. $({}^A R_C)^\top$ **changes the frame of reference**. That reversed arrow is still expressed in terms of $\{A\}$ but we need it in terms of $\{C\}$; $({}^A R_C)^\top = {}^C R_A$ implements that that change.

4 Right-handed axes and the right-hand rule

Every frame in this course is **right-handed**: $x \times y = z$ (and cyclically, $y \times z = x$, $z \times x = y$). This choice also fixes which rotation direction counts as *positive*. In every figure the axes are drawn colour-coded by the usual convention: the x -axis in **red**, the y -axis in **green**, and the z -axis in **blue**: xyz=RGB.

A positive rotation about an axis follows the **right-hand rule**: point your right thumb along the positive axis; the direction your fingers curl is the positive rotation direction. Viewed from the tip of $+z$ looking back towards the origin (i.e. z pointing straight out of the page at you), a positive rotation about z sends x towards y :

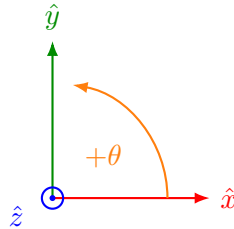


Figure 1. The right-hand rule for a positive rotation about z : with z pointing out of the page, $+\theta$ sends x towards y .

5 The rotation matrix, column by column

Each *column* of ${}^A R_B$ is one of $\{B\}$'s axes, expressed in $\{A\}$'s coordinates:

$${}^A R_B = \begin{bmatrix} | & | & | \\ {}^A \hat{x}_B & {}^A \hat{y}_B & {}^A \hat{z}_B \\ | & | & | \end{bmatrix},$$

where ${}^A \hat{x}_B$ is $\{B\}$'s x -axis unit vector, expressed in $\{A\}$'s coordinates (and similarly for ${}^A \hat{y}_B$, ${}^A \hat{z}_B$). For a “simple” rotation, one you can visualise directly, this means R can be written down *by inspection*: work out where each of $\{B\}$'s axes points, in $\{A\}$'s coordinates, and read those off as the columns.

Worked example: 60° about z

Take $\{B\}$ rotated 60° about $\{A\}$'s z -axis, with no other rotation. The rotation is *about* z , so $\{B\}$'s z -axis is unchanged: ${}^A \hat{z}_B = (0, 0, 1)$. The other two columns can be read straight off

Figure 2, where $\{A\}$ (solid) and $\{B\}$ (dashed) share the same origin and differ only by that 60° rotation; the dotted lines give $\hat{x}_B$'s coordinates $(\cos 60^\circ, \sin 60^\circ)$ in $\{A\}$:

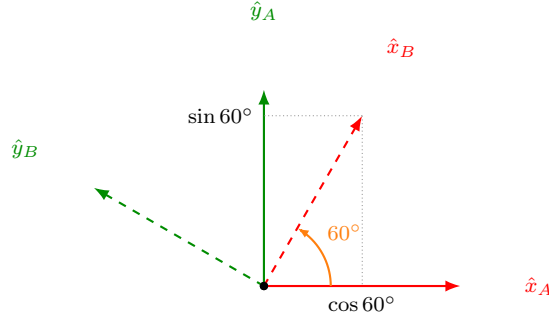


Figure 2. Reading ${}^A R_B$ by inspection. $\{A\}$ (solid) and $\{B\}$ (dashed) share an origin and differ by a 60° rotation about z ; the dotted lines give $\hat{x}_B = (\cos 60^\circ, \sin 60^\circ)$, and each of $\{B\}$'s axes, in $\{A\}$'s coordinates, is a column of ${}^A R_B$.

$${}^A R_B = \begin{bmatrix} \cos 60^\circ & -\sin 60^\circ & 0 \\ \sin 60^\circ & \cos 60^\circ & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0.5 & -0.866 & 0 \\ 0.866 & 0.5 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$\underbrace{\quad}_{\textcolor{red}{A}\hat{x}_B} \quad \underbrace{\quad}_{\textcolor{green}{A}\hat{y}_B} \quad \underbrace{\quad}_{\textcolor{blue}{A}\hat{z}_B}$

The braced columns are the read-off axes: $\textcolor{red}{A}\hat{x}_B$, $\textcolor{green}{A}\hat{y}_B$ and $\textcolor{blue}{A}\hat{z}_B$ are $\{B\}$'s x -, y -, and z -axes expressed in $\{A\}$'s coordinates.

To see rotation and translation together, extend the example: keep the same 60° rotation about z , but now also translate $\{B\}$ by ${}^A t_B = (4, 2, 0)$ relative to $\{A\}$. The dashed arrow below is that translation vector; the small arc at $\{B\}$'s origin, drawn against a dashed line parallel to $\{A\}$'s x -axis, is the same 60° rotation as before:

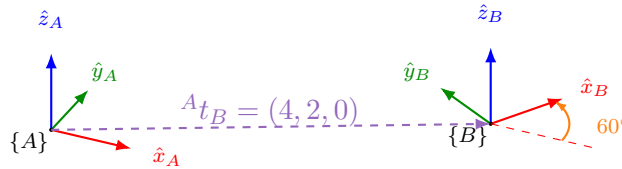


Figure 3. The same 60° rotation about z , now with a translation ${}^A t_B = (4, 2, 0)$ (dashed purple), giving the full pose of $\{B\}$ relative to $\{A\}$.

Combining both gives the complete homogeneous transform:

$${}^A T_B = \begin{bmatrix} 0.5 & -0.866 & 0 & 4 \\ 0.866 & 0.5 & 0 & 2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Recovering a rotation from points known in both frames

Often you cannot *see* the rotation, but you know the coordinates of two points in *both* frames: their coordinates in a local frame $\{B\}$, say from a drawing of a piece of equipment, and their coordinates in a workspace frame $\{A\}$, from surveying points in the robot's workspace as in Part B. That is enough to recover the rotation angle between the frames, in three steps.

1. Share an origin. Pick one of the two points as a common origin O , and call the other the second point P . Subtract O from P once in each frame:

$${}^B d = {}^B P - {}^B O, \quad {}^A d = {}^A P - {}^A O.$$

This removes the translation, leaving a single arrow d from O to P , with $\{A\}$ and $\{B\}$ now sharing an origin (Figure 4).

2. One arrow, two sets of coordinates. The displacement d , from the common origin to the other point P , is a single arrow in space: choosing a different frame does not move it. What changes is the set of coordinates that describe it. The coordinates in $\{B\}$ are ${}^Bd = (q_x, q_y)$ and the coordinates in $\{A\}$ are ${}^Ad = (d_x, d_y)$. Both sets are known.

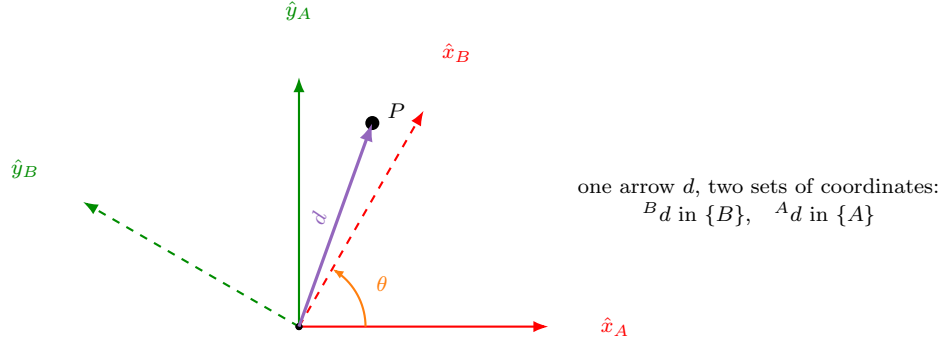


Figure 4. Recovering the rotation angle θ from one point. The point P is off both frames' axes, so the arrow d from the shared origin to P has two non-zero coordinates in each frame: Bd in the dashed frame $\{B\}$ and Ad in the solid frame $\{A\}$. The rotation angle θ between the frames is the angle from one set of coordinates to the other.

3. The angle from a dot product and a cross product. For a rotation about z the two sets of coordinates differ only by that rotation, ${}^Ad = R_z(\theta) {}^Bd$. So keep $\{A\}$'s axes from Figure 4, drop $\{B\}$'s, and plot *both* sets of coordinates on them (Figure 5): Ad is the arrow d itself, and Bd is the same length L , lying θ back from it. The angle between the two is the θ we want.

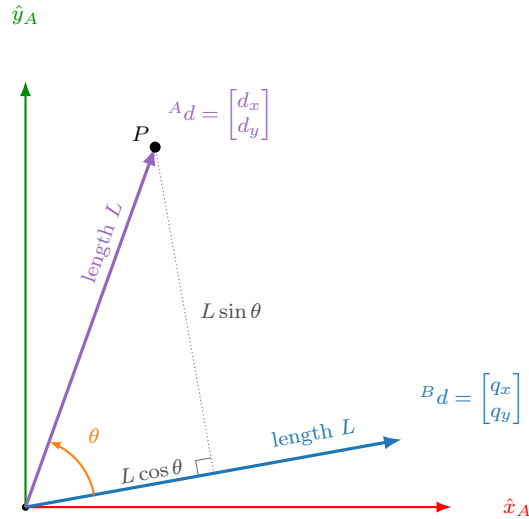


Figure 5. Where $\theta = \text{atan2}(\text{cross}, \text{dot})$ comes from. Figure 4 redrawn with $\{B\}$'s axes dropped and *both* sets of coordinates plotted on $\{A\}$'s: Ad is the arrow d itself, and Bd is the same length L , lying θ back from it. Dropping a perpendicular from the tip of Ad onto Bd splits it into a part $L \cos \theta$ along Bd and a part $L \sin \theta$ across it: the dot- and cross-product directions.

Two identities you already know for a pair of vectors give θ directly: the dot product is L times the projection of Ad onto Bd 's direction, and the (2-D) cross product is L times its perpendicular offset,

$${}^Bd \cdot {}^Ad = L^2 \cos \theta, \quad {}^Bd \times {}^Ad = L^2 \sin \theta,$$

the second being the z -component of the cross product, $q_x d_y - q_y d_x$ for two vectors in the xy -plane. Writing both out in coordinates and passing them to `atan2`, rather than dividing one by the other, keeps the sign of each and the common L^2 cancels automatically:

$$\theta = \text{atan2}\left(\underbrace{q_x d_y - q_y d_x}_{\text{cross}}, \underbrace{q_x d_x + q_y d_y}_{\text{dot}}\right).$$

The *on-axis* case is just a special case of this: if the point sits on $\{B\}$'s y -axis, its $\{B\}$ coordinates are ${}^B d = (0, 1)$, and for a 60° rotation its $\{A\}$ coordinates would be ${}^A d = (-0.866, 0.5)$. Feeding both coordinates into the general formula gives $\theta = \text{atan2}(0.866, 0.5) = 60^\circ$: exactly the numbers behind Part B's first example. Nor is the rule special to z : a rotation about x or y is read off the same way, as you will do for the button's -90° rotation about y in Part B. Part B's second example, where the point is *not* on an axis, uses the general formula directly.

Part B: The UR5 Assignment

Locating equipment and computing task poses

6 Determining equipment frame poses

For each piece of equipment we have the coordinates of at least two points in *both* the equipment's own frame and the UR5 (world) frame. From that information we can determine the pose of the equipment frame relative to the UR5 frame.

Let's work an example: finding the pose of the Rancilio (CM = coffee machine) frame relative to the UR5 (UR) frame. We have the following points (the key number from the points spreadsheet is listed alongside each):

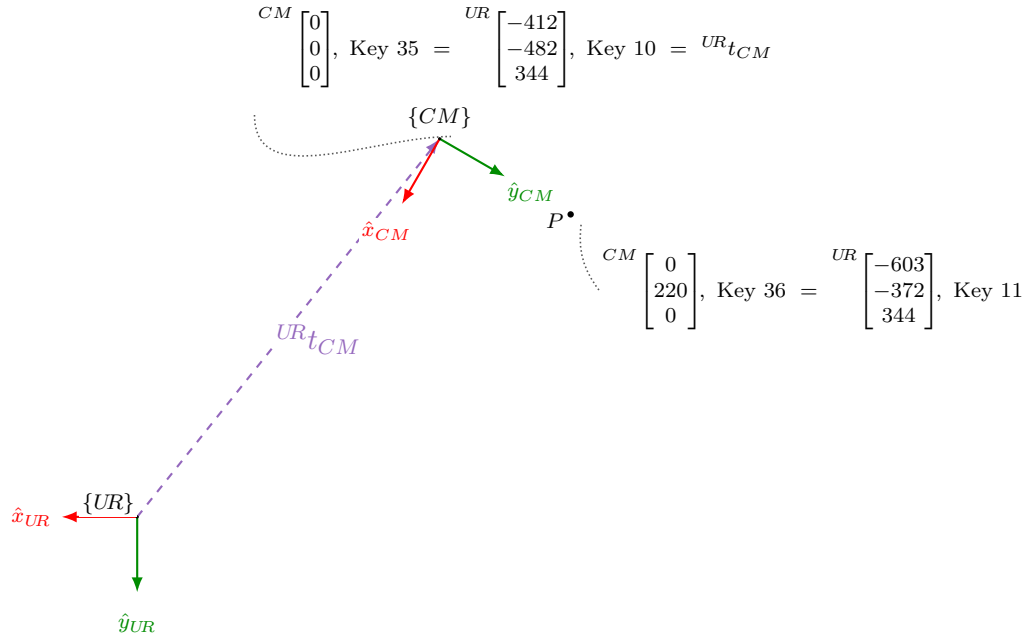


Figure 6. The two surveyed points for the coffee machine: the origin and a second point P , each known in both $\{CM\}$ and $\{UR\}$ coordinates (with their points spreadsheet key-numbers).

This is a simple translation and rotation about the z -axis, a planar (2-D) rotation. Focusing on just the rotation, put $\{UR\}$ (solid) and $\{CM\}$ (dashed, to tell them apart) at a common origin; we need the rotation angle θ between them.

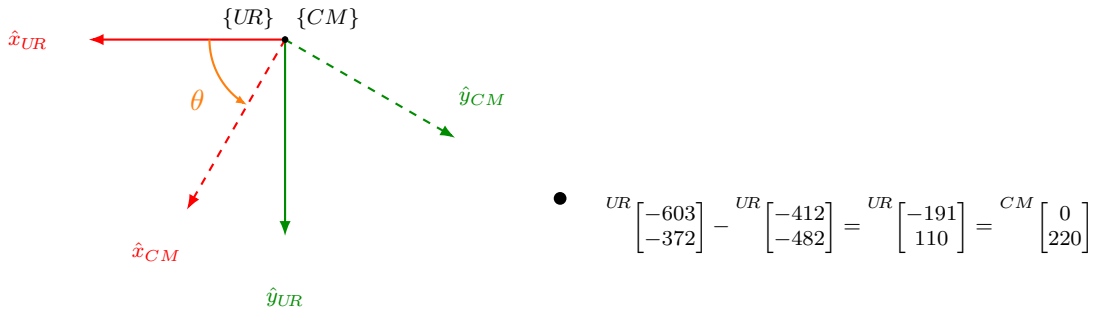


Figure 7. Recovering the rotation: $\{UR\}$ (solid) and $\{CM\}$ (dashed) placed at a common origin. The direction to the second point, in $\{UR\}$ coordinates, fixes the rotation angle θ between them.

The second point, P , is $(0, 220, 0)$ in $\{CM\}$ (Key 36): that is, 220 straight along $+y_{CM}$, so it lies on $\{CM\}$'s y -axis. Following step 1 of Part A, subtract the origin to get the displacement

from the origin to that point, in $\{UR\}$ coordinates, $(-603, -372) - (-412, -482) = (-191, 110)$. Normalising by its length, $\sqrt{191^2 + 110^2} = 220.4$, gives the direction of $\{CM\}$'s y -axis in $\{UR\}$ coordinates, $(-191/220.4, 110/220.4) = (-0.867, 0.499) \approx (-0.866, 0.5)$. (The length $220.4 \approx 220$ is a good sanity check: the two points are the same distance apart in both frames, as they must be, and the same survey tolerance accounts for the small step to the exact values.) That direction is not scaffolding to be discarded: by the column rule of Section 5, $\{CM\}$'s y -axis in $\{UR\}$ coordinates is exactly the second column of ${}^{UR}R_{CM}$, which is where $(-0.866, 0.5, 0)$ appears in the matrix below.

This is the on-axis special case from Part A because the point sits on $+\hat{y}_{CM}$. Therefore, applying the equation from Part A gives the rotation angle:

$$\theta = \text{atan2}(0.866, 0.5) = 60^\circ.$$

The equipment is therefore rotated 60° about z relative to the robot, with its origin at $\{UR\}$ coordinates $(-412, -482, 344)$ (Key 10). Assembling ${}^{UR}R_{CM} = R_z(60^\circ)$ with that translation gives the pose of the coffee machine relative to the robot:

$${}^{UR}T_{CM} = \begin{bmatrix} 0.5 & -0.866 & 0 & -412 \\ 0.866 & 0.5 & 0 & -482 \\ 0 & 0 & 1 & 344 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

A second example: the second point off the axes

The same idea handles a harder case, where the second point does *not* lie on one of the equipment frame's axes. Take a second piece of equipment, the $\{CS\}$ (cup stand) frame. We again have two points: its origin, which is $(0, 0, 0)$ in its own frame by definition, and a second point P known to sit at $\{CS\}$ coordinates $(0, 142, 90)$, with both points also known in $\{UR\}$ coordinates:

$$\underbrace{{}^{CS}(0, 0, 0) = {}^{UR}(-591.9, -215.4, 214.1)}_{\text{origin (Key 16 = Key 1)}}, \quad \underbrace{{}^{CS}(0, 142, 90) = {}^{UR}(-680.7, -72.4, 214.1)}_{\text{second point (Key 17 = Key 2)}}.$$

By inspection the equipment is mounted with its x -axis pointing straight up: that is a -90° rotation about y relative to $\{UR\}$ (call it the y -rotation), putting $\hat{x}_{CS}$ along $+\hat{z}_{UR}$ (Figure 8). What inspection does *not* give is the equipment's rotation about z (call it the z -rotation), and that is what the second point fixes.

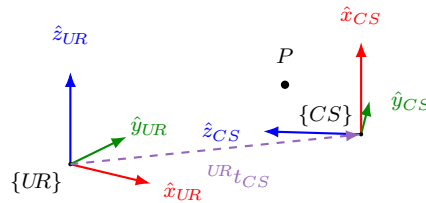


Figure 8. The $\{CS\}$ frame, mounted with its x -axis along $+\hat{z}_{UR}$ (a -90° y -rotation). The origin and a second point P are known in both $\{CS\}$ and $\{UR\}$ coordinates; P is not on any $\{CS\}$ axis.

The origin's $\{UR\}$ coordinates are the translation directly, ${}^{UR}t_{CS} = (-591.9, -215.4, 214.1)$. For the rotation we would like to reuse the planar method of Part A, but two things get in the way: the second point is not along a $\{CS\}$ axis (it has both a y and a z coordinate), and $\{CS\}$'s own z -axis is not $\hat{z}_{UR}$ (its x -axis is, here), so “rotation about z ” in $\{CS\}$'s own frame no longer means the z -rotation we actually want.

An intermediate upright frame. Introduce an intermediate frame $\{CS'\}$ at the same origin as $\{CS\}$ but with its z -axis aligned with $\hat{z}_{UR}$. $\{CS'\}$ and $\{CS\}$ differ only by the known y -rotation (rotating $\{CS'\}$ by -90° about its y -axis brings its z up onto $\hat{x}_{CS}$, Figure 9), so

$${}^{CS'}R_{CS} = R_y(-90^\circ), \quad {}^{UR}R_{CS} = {}^{UR}R_{CS'} {}^{CS'}R_{CS} = {}^{UR}R_{CS'} R_y(-90^\circ).$$

Now $\{CS'\}$ only differs from $\{UR\}$ by a pure rotation about z , ${}^{UR}R_{CS'} = R_z(\psi)$, exactly the planar situation of Part A. So we move the second point into $\{CS'\}$ and recover ψ there.

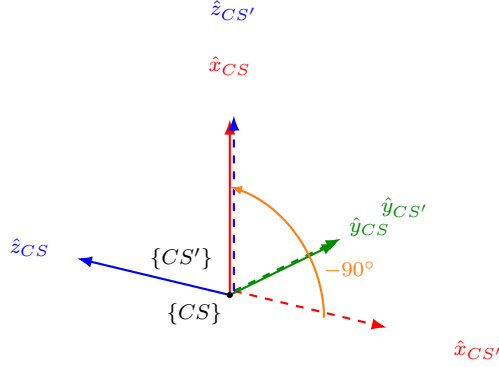


Figure 9. The $\{CS\}$ frame (solid, x up) and the intermediate frame $\{CS'\}$ (dashed, z up) at a common origin, related by the known -90° y -rotation. ($\{CS'\}$ is drawn a hair offset so the axes it shares with $\{CS\}$ stay visible.)

P 's $\{CS'\}$ coordinates follow from the known y -rotation:

$${}^{CS'}P = {}^{CS'}R_{CS} {}^{CS}P = R_y(-90^\circ) \begin{bmatrix} 0 \\ 142 \\ 90 \end{bmatrix} = \begin{bmatrix} -90 \\ 142 \\ 0 \end{bmatrix}.$$

Its z' coordinate is 0: the point lies in $\{CS'\}$'s xy -plane, at the same height as the origin, which is why the two surveyed points came out at the same height ($z = 214.1$) in $\{UR\}$.

Recovering the z -rotation. The problem is now planar, in the xy -plane. Because the origin sits at $(0, 0, 0)$ in $\{CS'\}$, P 's coordinates there are already the displacement from the origin to P , so dropping the zero z' coordinate gives ${}^{CS'}d = {}^{CS'}P$ as a 2-D vector, with components named q_x, q_y to match Part A's general formula. That is the Bd of Part A. The same displacement is known in $\{UR\}$ coordinates too, but there the origin is not at zero, so it has to be subtracted out as in step 1:

$${}^{CS'}d = \begin{bmatrix} q_x \\ q_y \end{bmatrix} = \begin{bmatrix} -90 \\ 142 \end{bmatrix}, \quad {}^{UR}d = \begin{bmatrix} -680.7 \\ -72.4 \end{bmatrix} - \begin{bmatrix} -591.9 \\ -215.4 \end{bmatrix} = \begin{bmatrix} -88.8 \\ 143.0 \end{bmatrix},$$

and ${}^{UR}R_{CS'} = R_z(\psi)$ maps one to the other, ${}^{UR}d = R_z(\psi) {}^{CS'}d$. The point is off the axes, so we use the general formula from Part A directly, the angle *from* the $\{CS'\}$ coordinates ${}^{CS'}d$ to the $\{UR\}$ coordinates ${}^{UR}d$:

$$\psi = \text{atan2}(q_x d_y - q_y d_x, q_x d_x + q_y d_y) = \text{atan2}(-260.4, 28298) = -0.5^\circ.$$

Both vectors have length ≈ 168 (168.1 and 168.3 respectively; the small mismatch is real survey precision, the same tolerance as the coffee machine example's $220.4 \approx 220$), as they must, since it is the same displacement expressed in two frames.

Here $\psi \approx -0.5^\circ$, close enough to zero that the cup stand's z -rotation relative to $\{UR\}$ is, in practice, negligible: this unit turns out to be mounted essentially straight, differing from $\{UR\}$ by only the known y -rotation. That could not have been assumed in advance, which is exactly

why the general method was needed, nothing in the setup told us the z -rotation would come out this small until the second point's coordinates were actually used.

Assembling the z -rotation with the known y -rotation gives the rotation, and with the origin, the full pose:

$${}^{UR}R_{CS} = R_z(-0.5^\circ) R_y(-90^\circ),$$

$${}^{UR}T_{CS} = \begin{bmatrix} 0 & 0 & -1 & -591.9 \\ 0 & 1 & 0 & -215.4 \\ 1 & 0 & 0 & 214.1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The recipe generalises the first example: take two points known in both frames, use the origin as the translation, and if the second point is off the axes, rotate it into an upright intermediate frame so the leftover rotation is a plain rotation about z .

7 Task poses: commanding the robot's tool

Locating the equipment is only half the job; the assignment then asks you to drive the robot to a *task* pose defined relative to that equipment, here pressing a button on the coffee machine. To do this, we need to calculate the pose of the robot tool centre point in the UR frame that will bring the tool into contact with the button. We need to select and place frames appropriately to achieve this task: $\{BTN\}$ on the button, $\{MT\}$ the Mazzer tool and $\{TCP\}$ the tool centre point the robot actually commands.

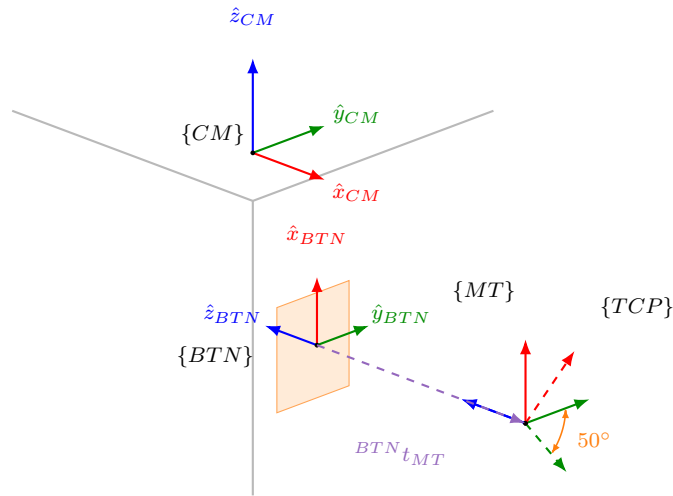


Figure 10. The task scene. The coffee machine frame $\{CM\}$ on its top surface and the button on the front face. Three tool-side frames connect to it: $\{BTN\}$ on the button, $\{MT\}$ the Mazzer tool, and $\{TCP\}$ the tool centre point the robot commands.

The button frame, by inspection

$\{BTN\}$ is $\{CM\}$ rotated -90° about $\hat{y}_{CM}$, then translated to the button location, ${}^{CM}t_{BTN} = (50, 38, -30)$. Write ${}^{CM}R_{BTN}$ by inspection (Part A):

$${}^{CM}R_{BTN} = \begin{bmatrix} 0 & 0 & -1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}, \quad {}^{CM}t_{BTN} = \begin{bmatrix} 50 \\ 38 \\ -30 \end{bmatrix}.$$

The button's pose in the robot frame is then one multiplication, ${}^{UR}T_{BTN} = {}^{UR}T_{CM} {}^{CM}T_{BTN}$:

$${}^{UR}T_{BTN} = \begin{bmatrix} 0 & -0.866 & -0.5 & -419.909 \\ 0 & 0.5 & -0.866 & -419.699 \\ 1 & 0 & 0 & 314 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The tool, and what the robot actually controls

The robot does not command the button's pose; it commands its own **tool centre point** (TCP). Two frames connect the tool to the button (both drawn in the figure above, with horizontal z -axes, since the approach is horizontal):

- $\{MT\}$, the Mazzer tool, has the *same orientation* as $\{BTN\}$ but sits back along the approach direction: ${}^{BTN}R_{MT} = I$, ${}^{BTN}t_{MT} = (0, 0, -103)$.
- $\{TCP\}$ shares $\{MT\}$'s origin but is rotated about its z -axis: ${}^{TCP}R_{MT} = R_z(-50^\circ)$.

Solving for the commanded pose. There are two routes from $\{UR\}$ to the button, and they must be equal for the tip of the Mazzer tool to press the button: via the tool, or via the equipment,

$$\underbrace{{}^{UR}T_{TCP} {}^{TCP}T_{MT} {}^{MT}T_{BTN}}_{\text{robot side}} = \underbrace{{}^{UR}T_{CM} {}^{CM}T_{BTN}}_{\text{equipment side}}$$

Both products describe the same thing, ${}^{UR}T_{BTN}$, so they are equal. On each side the matching inner labels cancel, exactly as in Section 3:

$$\begin{array}{c} {}^{UR}T_{TCP} \xrightarrow{\text{TCP}} T_{MT} \xrightarrow{\text{MT}} T_{BTN} \end{array} = \begin{array}{c} {}^{UR}T_{CM} \xrightarrow{\text{CM}} T_{BTN} \end{array}$$

Each side collapses to ${}^{UR}T_{BTN}$.

Everything in the identity is known except ${}^{UR}T_{TCP}$, the pose we must send the robot. Solve for it by moving the two right-most factors on the left over to the right, each becoming an inverse (see “Inverting a transform” above). Be **very careful** about the order, because matrix multiplication is not commutative:

$${}^{UR}T_{TCP} = {}^{UR}T_{CM} {}^{CM}T_{BTN} ({}^{MT}T_{BTN})^{-1} ({}^{TCP}T_{MT})^{-1} = {}^{UR}T_{CM} {}^{CM}T_{BTN} {}^{BTN}T_{MT} {}^{MT}T_{TCP}.$$

Evaluating with the numbers above:

$${}^{UR}T_{TCP} = \begin{bmatrix} -0.663 & -0.557 & -0.5 & -368.409 \\ 0.383 & 0.321 & -0.866 & -330.498 \\ 0.643 & -0.766 & 0 & 314 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

This 4×4 matrix is the pose you command the UR5's TCP to, to press the button.